



Modernise & Migrate: DWER

TravelReporting — Platform, IT Ops & SecOps/Security Track

Welcome to **Day 2 (Build & Migrate)** and **Day 3 (Harden & Deliver)** of the Application Modernisation OpenHack for the Department of Water and Environmental Regulation (DWER). Day 1 (Discovery & Orientation) is already complete.

Over the next two days you will design and build the Azure platform that hosts the modernised TravelReporting application — provisioned as Bicep infrastructure-as-code, secured with Key Vault, monitored with Azure Monitor and Application Insights, protected with Microsoft Defender for Cloud and GitHub Advanced Security, and connected through a hub-spoke network with Azure Firewall.

Why this track?

Platform Engineers, SREs, IT Operations, and Security Engineers are the people who make modernised applications safe to run in production. While the App Dev track builds the application, your track builds everything around it — the network, the identity plumbing, the secrets management, the observability stack, the security controls, and the runbooks that keep it running at 3am. This track treats **platform and security as a single discipline**: infrastructure that is not secure by default is not production-ready, and security controls that are not automated are not controls at all.

Your Tools

GitHub Copilot Chat Modes

Mode	When to use	Think of it as...
 Ask	Understanding Azure service behaviour, reviewing Bicep for security issues, explaining KQL query results, threat modelling	Your Azure architect on call
 Plan	Designing hub-spoke network topology, planning secrets migration, designing alert strategy before writing KQL	Your tech lead writing the runbook spec
 Agent	Generating Bicep templates, writing KQL queries, scaffolding runbooks, creating smoke-test scripts, generating IaC security fixes	Your platform engineer who never sleeps

Rule of thumb: Ask → Plan → Agent. Understand the threat before you plan the control, plan the control before you automate it.

Azure & Security Accelerators

Tool	When it helps
Azure MCP Server	Live Azure resource inspection, provisioning validation, KQL log queries, Defender for Cloud recommendations, RBAC queries — without leaving VS Code
Azure Skills for GitHub Copilot	One-liner commands: <code>@azure-prepare</code> , <code>@azure-deploy</code> , <code>@azure-rbac</code> , <code>@azure-diagnostics</code> , <code>@azure-compliance</code> , <code>@azure-cost</code> , <code>@azure-resource-visualizer</code> — invoke Azure operations via Copilot
Bicep MCP (<code>mcp_bicep_*</code>)	Validate generated Bicep against best practices, check secure-baseline compliance, catch insecure defaults before deployment
Awesome Copilot	Reusable <code>.prompt.md</code> and <code>.instructions.md</code> from github.com/github/awesome-copilot — grab IaC, KQL, runbook, and security-review prompt templates

Tool	When it helps
GitHub Advanced Security / CodeQL	SAST on the migrated .NET code, secret scanning, push protection, Dependabot alerts — shift-left security in the PR pipeline
Microsoft Defender for Cloud	Cloud Security Posture Management (CSPM), Defender plans for App Service and Azure SQL, regulatory compliance dashboard, security recommendations

The Workload & Threat Surface

What is being operationalised

TravelReporting is a travel approval and reporting system for DWER staff. When modernised (by the App Dev track), it becomes:

Component	Details
API	.NET 8 minimal API (<code>TravelReporting.Api</code>) on Azure App Service
Frontend	React + Vite or Blazor Server (<code>TravelReporting.Web</code>) on Azure App Service
Database	Azure SQL (migrated from Oracle <code>TRRE</code> schema — <code>TRRE-Oracle/TABLE.sql</code>)
Scale	Single-department internal tool (~200–500 concurrent users at peak travel-approval periods)
Data sensitivity	Staff PII (names, emails, phone numbers), financial cost-centre data, travel approval chains

Current operational gaps

Gap	Evidence in workspace
No IaC	No <code>.bicep</code> , <code>.tf</code> , <code>Dockerfile</code> , or <code>docker-compose.yml</code> anywhere in the workspace
No observability	No logging framework, no health endpoint, no telemetry in <code>TravelReporting/</code>
No secrets management	Hardcoded ODBC datasource name and email addresses in <code>TravelReporting/application.cfm</code>
No CI/CD	No pipeline configuration of any kind
No network controls	IIS-hosted, no VNet, no firewall, no WAF configuration
Legacy identity	Windows NT integrated auth (<code>CGI.AUTH_USER</code>) in <code>TravelReporting/application.cfm</code> — no MFA, no conditional access

Current threat surface

Threat	Evidence
SQL injection	Every <code><cfquery></code> block in <code>TravelReporting/Travel/Edit.cfm</code> , <code>TravelReporting/Travel/Save.cfm</code> , <code>TravelReporting/Search/Select.cfm</code> uses raw SQL string concatenation — classic injection surface
Hardcoded credentials/addresses	<code>TravelReporting/application.cfm</code> contains <code>mailto="peter.kempf@dwer.wa.gov.au"</code> — staff PII in source code
No TLS enforcement	No HTTPS-only configuration found in any source file
No input validation	No server-side validation layer; ColdFusion accepts any form input directly into SQL

Threat	Evidence
No audit logging	No record of who approved, modified, or deleted travel records
PII in travel records	TRRE-Oracle/TABLE.sql — TRRE.TRAVEL table contains FIRST_NAME , SURNAME , PERSON , WORK_EMAIL

Key reference files for this track

File	Relevance to platform/security work
TravelReporting/application.cfm	NT auth pattern to replace; hardcoded ODBC and email addresses to move to Key Vault
TravelReporting/Travel/Save.cfm	SQL injection surface — verify EF Core migration eliminates all raw SQL
TRRE-Oracle/TABLE.sql	Source DDL to drive Azure SQL schema design and data-protection decisions
TRRE-Oracle/VIEW.sql	Cross-schema dependencies (COMA , COSM , BUSY) to stub or migrate

Agenda

Day	Phase	Topic	Time
Day 2	1	Secure-by-Default Bicep IaC Generation	60 min
Day 2	2	Identity, Key Vault & Environment Configuration	60 min
Day 2	3	Azure Monitor, App Insights & Security Alert Rules	60 min

Day	Phase	Topic	Time
Day 2	4	Health Check & Security Smoke Scripts	60 min
Day 3	5	Network Hardening: Hub-Spoke + Azure Firewall	60 min
Day 3	6	Incident Response Runbooks & SecOps Playbooks	60 min
Day 3	7	Defender for Cloud, GitHub Advanced Security & WAF Review	60 min
Day 3	8	Operational & Security Dashboards + Handover Documentation	60 min

The Phases

Phase 1 — Secure-by-Default Bicep IaC Generation

Agent mode + Bicep MCP

The TravelReporting application currently has no infrastructure-as-code of any kind. Files were presumably copied to an on-premises IIS server. Your first job is to give the modernised application a reproducible, auditable, secure-by-default Azure foundation — described entirely in Bicep so that every resource is created the same way every time, and so that security controls are baked in rather than bolted on.

Your Challenge

Generate Bicep templates that provision the full Azure footprint for `TravelReporting.Api` and `TravelReporting.Web`, with security defaults enabled from the first deployment.

Things to Explore/Do

- Use Agent mode to scaffold the Bicep module structure:

```

infra/
  main.bicep          # orchestration
  modules/
    app-service.bicep # App Service Plan + Web App
    sql.bicep         # Azure SQL Server + Database
    keyvault.bicep    # Key Vault
    monitoring.bicep  # Log Analytics + App Insights
    network.bicep     # Hub VNet, Spoke VNet, peering (Phase 5)

```

- Ask Copilot: "Generate a Bicep module for Azure App Service that enforces HTTPS-only, sets minimum TLS version to 1.2, enables system-assigned Managed Identity, configures diagnostic settings to send all logs to a Log Analytics workspace, and disables FTP and SCM public access."
- Ask Copilot: "Generate a Bicep module for Azure SQL Server and Database that disables public network access, enables Microsoft Entra-only authentication, enables Advanced Threat Protection, and configures audit logs to the Log Analytics workspace."
- Ask Copilot: "Generate a Bicep module for Azure Key Vault with soft-delete enabled (90 days), purge protection enabled, RBAC authorisation model (not access policies), network ACLs set to deny by default, and diagnostic settings to Log Analytics."
- After generating each module, validate it with the Bicep MCP: use `mcp_bicep_get_best_practices` before deploying, and cross-check against any `mcp_bicep_validate_*` tools available.
- Ask Copilot: "Review my App Service Bicep module and list any security hardening settings I have missed, referencing the Azure Security Benchmark for App Service."
- Use `@azure-prepare` to verify your Azure subscription has the required resource providers registered (`Microsoft.Web` , `Microsoft.Sql` , `Microsoft.KeyVault` , `Microsoft.Insights` , `Microsoft.Network`).
- Parameterise the templates for `dev` , `test` , and `prod` environments using Bicep parameter files (`main.dev.bicepparam` , `main.prod.bicepparam`).

What Done Looks Like

- ☐ `infra/main.bicep` exists and references all required modules
- ☐ App Service module enforces HTTPS-only, TLS 1.2+, and disables FTP

- ☐ Azure SQL module has public network access disabled
- ☐ Key Vault module has soft-delete and purge protection enabled, RBAC model
- ☐ All resources send diagnostic logs to a shared Log Analytics workspace
- ☐ Bicep MCP validation passes with no critical security findings
- ☐ Parameter files exist for at least `dev` and `prod` environments

🌟 **Stretch Goal** — Add a Bicep template for Azure Front Door or Azure Application Gateway with WAF policy (OWASP 3.2 rule set) in front of the App Service, replacing direct public access to the App Service origin.

🛑 GIT CHECKPOINT

```
git add infra/ && git commit -m "phase-1: secure-by-default Bicep IaC - App"
```

Phase 2 — Identity, Key Vault & Environment Configuration

🤖 Agent mode

The original *TravelReporting* application stores its database connection string as a plain ODBC datasource name and puts staff email addresses directly in source code (`TravelReporting/application.cfm`). In the modernised system, every secret lives in Key Vault, every service identity is a Managed Identity, and no human credentials are embedded in any configuration file.

Your Challenge

Wire up Azure Key Vault as the secrets store for `TravelReporting.Api` , remove all hardcoded credentials from configuration, and define the identity strategy for both human users and service-to-service authentication.

Things to Explore/Do

Key Vault secrets (all teams):

- Use Agent mode: "Generate the Azure CLI or Bicep commands to store the following secrets in Key Vault: SQL connection string, App Insights instrumentation key, and notification email address (replacing the hardcoded address in `TravelReporting/application.cfm`)."

- Add Key Vault secret references to the App Service app settings in your Bicep: `@Microsoft.KeyVault(SecretUri= ... )` — this avoids the application needing any Key Vault SDK code.
- Verify no secrets appear in `infra/*.bicep` or `infra/*.bicepparam` files — use GitHub Advanced Security secret scanning to confirm.

Managed Identity (recommended):

- Use Agent mode: *"Add a system-assigned Managed Identity to the App Service Bicep module, then add a Key Vault Secrets User role assignment scoped to the Key Vault, so the App Service can read secrets without a client secret or connection string."*
- Ask Copilot: *"Show me how to use DefaultAzureCredential in a .NET 8 application so it works both locally (via Azure CLI login) and in production (via Managed Identity) without changing code."*

Alternative — if Managed Identity is not available in your hackathon environment:

- Use Key Vault access policies with a service principal client ID and secret stored in App Service application settings.
- Ask Copilot: *"Generate a PowerShell script to create an Entra ID app registration, assign it Key Vault Secrets User role, and output the client ID and secret for use in App Service configuration — with a note that the secret must be rotated after the hackathon."*

Identity for human users:

- **Entra ID (recommended):** Ask Copilot to generate the Bicep and `appsettings.json` changes needed to enable Microsoft Entra ID authentication on the App Service (Easy Auth), replacing the NT `CGI.AUTH_USER` pattern from `TravelReporting/application.cfm`.
- **Fallback (no Entra access):** Use App Service built-in authentication disabled, with a static API key in Key Vault for the hackathon session. Document that production must use Entra ID.

RBAC least-privilege review:

- Use `@azure-rbac` to list the current role assignments on your resource group and identify any over-privileged identities.
- Ask Copilot: *"What is the minimum RBAC role needed for the App Service Managed Identity to read from Azure SQL using Entra authentication, and how do I create the SQL user for the managed identity?"*

What Done Looks Like

- ☐ All secrets are stored in Key Vault — no plaintext credentials in any config file or Bicep template
- ☐ App Service reads secrets via Key Vault references or Managed Identity + `DefaultAzureCredential`
- ☐ GitHub Advanced Security secret scanning shows zero findings on the repository
- ☐ RBAC assignments follow least-privilege (no Owner or Contributor on the Key Vault for the App Service identity)
- ☐ Identity strategy is documented: which option was chosen (Managed Identity / service principal / Easy Auth) and why

☀️ **Stretch Goal** — Enable Key Vault soft-delete recovery and configure a Key Vault diagnostic alert that fires if a secret is deleted or purged, sending a notification to the operations email stored in Key Vault.

|| GIT CHECKPOINT

```
git add infra/ .github/ && git commit -m "phase-2: Key Vault wired up - secr
```

Phase 3 — Azure Monitor, App Insights & Security Alert Rules

💬 Ask + 🤖 Agent mode

The current TravelReporting system has no observability at all — if it fails, nobody knows until a user calls the helpdesk. The modernised system must surface both operational failures and security events automatically. This phase defines what "something is wrong" looks like — both from a reliability perspective and a security perspective — and turns those definitions into KQL alert rules.

Your Challenge

Define SLIs/SLOs for the TravelReporting application, write KQL-based alert rules for top operational and security failure modes, and configure actionable Azure Monitor alerts (no alert fatigue).

Things to Explore/Do

Operational observability:

- Ask Copilot: *"What SLIs and SLOs are appropriate for an internal government travel approval application like TravelReporting? Suggest availability, latency, and error-rate targets with rationale."*
- Use Agent mode: *"Write a KQL query for Azure Monitor / App Insights that alerts when the TravelReporting.Api HTTP 5xx error rate exceeds 5% over a 5-minute window."*
- Write KQL alert rules for:
 - App Service CPU > 80% for 10 minutes
 - Azure SQL DTU > 85% for 5 minutes
 - `POST /travels` endpoint p95 latency > 3 seconds
 - App Service plan memory > 90%
- Add an App Insights availability test (standard URL ping) targeting `GET /health` on the App Service.
- Browse github.com/github/awesome-copilot for reusable KQL prompt templates and adapt them for the TravelReporting workload.

Security alerting:

- Use Agent mode: *"Write a KQL query for Azure Monitor that detects more than 5 failed login attempts from the same IP address within 10 minutes on the TravelReporting App Service authentication logs."*
- Write KQL security alert rules for:
 - Azure SQL firewall rule added or modified (Azure Activity Log)
 - Key Vault secret accessed by an identity that is not the App Service Managed Identity
 - App Service TLS minimum version changed (Azure Activity Log)
 - Any resource in the TravelReporting resource group has a public network access rule added
- Use `#mcp_azure_query_logs` to validate your KQL against the real Log Analytics workspace before finalising alert rules.
- Use `@azure-diagnostics` to confirm that all resources are sending logs (verify no diagnostic setting gaps from Phase 1).

Alert routing:

- Ask Copilot to generate an Azure Monitor Action Group Bicep module that sends operational alerts to an operations email and security alerts to a security email — both stored as Key Vault secrets, not hardcoded.
- Define alert severity: Sev 0 (service down) → Sev 4 (informational). Avoid Sev 3/4 in production — they become noise.

What Done Looks Like

- ☐ At least 4 operational KQL alert rules are defined and attached to action groups
- ☐ At least 4 security KQL alert rules are defined (login failures, Key Vault access anomaly, firewall changes, public access changes)
- ☐ An App Insights availability test targets `GET /health`
- ☐ SLI/SLO targets are documented (availability %, latency p95, error rate threshold)
- ☐ Alert rules are in Bicep (`infra/modules/monitoring.bicep`) — not created manually in the portal
- ☐ No notification email addresses are hardcoded in Bicep — they reference Key Vault secrets

🌟 **Stretch Goal** — Use `@azure-resource-visualizer` to generate an Azure Monitor Workbook that shows all alert states in a single dashboard, so the on-call engineer can see the full health picture at a glance.

Phase 4 — Health Check & Security Smoke Scripts

Agent mode

Deployments are only as good as the verification that follows them. Manual click-testing is not a deployment gate. This phase produces automated scripts — suitable for running in the Azure DevOps pipeline as a post-deploy stage — that confirm the application is alive, its dependencies are reachable, and its security configuration has not drifted.

Your Challenge

Generate PowerShell smoke-test and security-check scripts that can run as a post-deploy pipeline stage in Azure DevOps to verify both operational health and security posture after every deployment.

Things to Explore/Do

Operational smoke tests:

- Use Agent mode: *"Generate a PowerShell script that: (1) calls GET /health on the TravelReporting App Service and asserts HTTP 200, (2) calls GET /travels and asserts HTTP 200 with a non-empty JSON body, (3) calls POST /travels with a minimal valid payload and asserts HTTP 201, (4) prints PASS/FAIL for each check with timestamps."*
- Add a dependency probe that verifies the App Service can reach Azure SQL (via a test connection endpoint or the `/health` route if it includes DB connectivity).
- Add a Key Vault connectivity probe: verify the App Service can read the SQL connection string secret using `az keyvault secret show`.

Security smoke checks:

- Use Agent mode: *"Generate a PowerShell script that checks: (1) the App Service enforces HTTPS (rejects HTTP with 301), (2) the response includes Strict-Transport-Security header, (3) the response includes X-Content-Type-Options header, (4) the App Service FTP is disabled, (5) the Azure SQL public network access is Disabled. Print PASS/FAIL for each."*
- Add a check that verifies no resource in the TravelReporting resource group has a public IP directly attached to a database.
- Add a secret-scan check: run `git log --all --full-history --` `TravelReporting/application.cfm` and look for any commit that introduced the `@water.wa.gov.au` or `@dwer.wa.gov.au` email addresses — verify they are no longer present in the current HEAD.
- Ask Copilot: *"Generate a PowerShell script using the Azure CLI that checks the RBAC assignments on the TravelReporting Key Vault and flags any identity with Owner or Contributor that is not a known admin service principal."*

Pipeline integration:

- Add a `smoke-test` stage to `azure-pipelines.yml` (from the App Dev track Phase 5) that runs these scripts after the deployment stage, with a `dependsOn: Deploy` condition and a `failOnStderr: true` setting.
- Confirm the scripts exit with a non-zero code on any FAIL — so the pipeline gate works correctly.

What Done Looks Like

- ☐ Operational smoke script covers at least: `/health`, `/travels` GET, `POST /travels`

- ☐ Security smoke script covers at least: HTTPS enforcement, HSTS header, FTP disabled, SQL public access disabled
- ☐ RBAC drift check script exists and flags unexpected Owner/Contributor assignments
- ☐ All scripts exit non-zero on failure (pipeline-compatible)
- ☐ Scripts are committed to the repository under `scripts/smoke/`
- ☐ A `smoke-test` stage exists in `azure-pipelines.yml` with `dependsOn: Deploy`

☀️ **Stretch Goal** — Add an SBOM generation step to the pipeline using `dotnet sbom-tool` and publish the SBOM as a pipeline artefact. Add a check that verifies no SBOM component has a known Critical CVE via OSV.dev or the GitHub Advisory Database.

II GIT CHECKPOINT

```
git add scripts/ azure-pipelines.yml && git commit -m "phase-4: smoke tests"
```

Phase 5 — Network Hardening: Hub-Spoke + Azure Firewall

📋 Plan + 🧑 Agent mode

The current TravelReporting application sits on an IIS server with no defined network boundary. The Azure target architecture uses a hub-spoke topology: a hub VNet containing Azure Firewall for centralised outbound control, and a spoke VNet for the TravelReporting workload with private endpoints for Azure SQL and Key Vault. No database or Key Vault traffic should ever traverse the public internet.

Your Challenge

Design and implement the Bicep IaC for a hub-spoke network topology with Azure Firewall, private endpoints for Azure SQL and Key Vault, App Service VNet Integration, and firewall rules appropriate for the TravelReporting workload.

Things to Explore/Do

- In Plan mode, ask Copilot: "Design a hub-spoke VNet topology for TravelReporting on Azure App Service. The hub contains Azure Firewall and a Bastion host. The spoke contains the App

Service VNet integration subnet, private endpoints for Azure SQL and Key Vault, and no public IP on any PaaS resource. Show the IP addressing scheme and the traffic flows."

- Use Agent mode to generate `infra/modules/network.bicep` :
 - Hub VNet (e.g. `10.0.0.0/16`) with:
 - `AzureFirewallSubnet` (`10.0.1.0/24`)
 - `AzureBastionSubnet` (`10.0.2.0/24`)
 - Spoke VNet (e.g. `10.1.0.0/16`) with:
 - `appservice-integration-subnet` (`10.1.1.0/24`) — delegated to `Microsoft.Web/serverFarms`
 - `private-endpoints-subnet` (`10.1.2.0/24`)
 - VNet peering (hub ↔ spoke, both directions)
 - Private DNS zones: `privatelink.database.windows.net` , `privatelink.vaultcore.azure.net`
 - Private endpoint for Azure SQL linked to `private-endpoints-subnet`
 - Private endpoint for Key Vault linked to `private-endpoints-subnet`
 - App Service VNet Integration linked to `appservice-integration-subnet`
- Ask Copilot: *"Generate Azure Firewall application and network rules appropriate for a .NET 8 App Service that connects to Azure SQL and Key Vault via private endpoints, fetches NuGet packages during deployment, and calls Microsoft Entra ID for token validation. Block all other outbound traffic."*
- Set `publicNetworkAccess: Disabled` on the Azure SQL and Key Vault Bicep modules (update the modules from Phase 1).
- Ask Copilot: *"What DNS resolution changes are required when moving Azure SQL and Key Vault to private endpoints? Generate the Azure Private DNS zone and A-record Bicep."*
- Use `#mcp_azure_list_resources` to verify the private endpoints are correctly provisioned and DNS resolution is working after deployment.
- Use `@azure-diagnostics` to confirm Azure Firewall logs are flowing to the shared Log Analytics workspace.

What Done Looks Like

- ☐ Hub VNet and Spoke VNet are deployed via Bicep with correct subnets
- ☐ Azure Firewall is deployed in the hub with application and network rules

- ☐ Private endpoints exist for Azure SQL and Key Vault in the spoke
- ☐ Private DNS zones are configured and linked to the spoke VNet
- ☐ App Service VNet Integration is configured
- ☐ `GET /health` on the App Service still returns 200 after network lockdown
- ☐ Direct public access to Azure SQL and Key Vault is blocked (verified by smoke script from Phase 4)

🌟 **Stretch Goal** — Add an Azure DDoS Network Protection plan to the hub VNet and configure a DDoS alert rule in Azure Monitor that fires when a DDoS attack is detected on the App Service public IP.

Phase 6 — Incident Response Runbooks & SecOps Playbooks

Agent mode

The TravelReporting application holds PII and financial data. When something goes wrong — whether a service outage or a security incident — the operations and security teams need a clear, tested procedure. Runbooks written after an incident are too late. This phase uses Copilot to generate both operational runbooks for the top likely service failures and SecOps playbooks for the top likely security incidents, including auto-remediation samples.

Your Challenge

Generate operational runbooks for the top 5 likely service failures and SecOps playbooks for the top 3 likely security incidents, with auto-remediation scripts where appropriate.

Things to Explore/Do

Operational runbooks — generate a runbook for each of the following, stored under `runbooks/ops/` :

1. **App Service unresponsive** — steps to check App Insights live metrics, restart the app, check Key Vault connectivity, check Azure SQL connectivity, escalate
2. **Azure SQL high DTU / slow queries** — steps to query `sys.dm_exec_requests` , identify blocking queries, check index recommendations, scale up the SKU
3. **Deployment failure** — steps to check Azure DevOps pipeline logs, roll back to the previous slot, verify smoke tests, re-trigger deployment

4. **Key Vault secret rotation** — steps to generate a new secret, store it in Key Vault, trigger an App Service configuration refresh, verify with the health endpoint
5. **Disk / storage quota alert** — steps for App Service file system usage, Azure SQL storage, Log Analytics workspace retention

Use Agent mode: *"Generate a Markdown runbook for 'App Service unresponsive' for the TravelReporting application. Include: symptoms, immediate triage steps using Azure CLI and App Insights KQL, resolution steps, rollback procedure, and post-incident actions."*

SecOps playbooks — generate a playbook for each, stored under `runbooks/secops/` :

1. **Compromised credential / stolen Entra ID token** — invalidate sessions, revoke refresh tokens via Entra ID, audit sign-in logs with KQL, rotate affected secrets in Key Vault, notify security team
2. **Secret exposed in source code** — immediately rotate the secret in Key Vault, revoke any tokens that used it, run `git filter-repo` to remove from history, enable push protection to prevent recurrence
3. **Suspicious sign-in / anomalous access to travel records** — KQL to identify affected records, suspend the user account, preserve logs for forensics, notify the data custodian, assess whether a data breach notification is required under WA privacy legislation

Use Agent mode: *"Generate a PowerShell auto-remediation script that: when triggered by an Azure Monitor alert for 'Key Vault secret accessed by unexpected identity', automatically revokes the identity's Key Vault Secrets User role assignment using the Azure CLI, logs the action to Log Analytics, and sends an email notification."*

- Ask Copilot: *"Generate a Logic App workflow (in JSON) that triggers on a Defender for Cloud security alert for the TravelReporting resource group and creates a GitHub issue in the repository with the alert details, severity, and a link to the affected resource."*

What Done Looks Like

- ☐ 5 operational runbooks exist under `runbooks/ops/` in Markdown
- ☐ 3 SecOps playbooks exist under `runbooks/secops/` in Markdown
- ☐ At least 1 auto-remediation script exists (PowerShell or Logic App JSON)
- ☐ Each runbook references the specific Azure resources from this workload (App Service name, Key Vault name, Log Analytics workspace)
- ☐ The secret-rotation runbook references the Key Vault secrets created in Phase 2

☀️ **Stretch Goal** — Create an Azure DevOps wiki page (or a `docs/runbook-index.md`) that lists all runbooks and playbooks with links, owner, last tested date, and the alert rule that triggers each one.

🛑 GIT CHECKPOINT

```
git add runbooks/ && git commit -m "phase-6: operational runbooks and SecOps"
```

Phase 7 — Defender for Cloud, GitHub Advanced Security & WAF Review

💬 Ask + 🤖 Agent mode

Security posture management is not a one-time activity — it is a continuous signal. This phase enables the security tools that will monitor the TravelReporting platform ongoing: Defender for Cloud for cloud posture and threat protection, and GitHub Advanced Security for supply-chain and code security. It concludes with a Well-Architected Framework self-assessment weighted on the Security pillar.

Your Challenge

Enable and configure Microsoft Defender for Cloud and GitHub Advanced Security on the TravelReporting workload, triage the initial findings, and complete a Security pillar WAF self-assessment.

Things to Explore/Do

Microsoft Defender for Cloud:

- Use Agent mode: "Generate Bicep to enable the following Defender for Cloud plans on the TravelReporting subscription: Defender for App Service, Defender for Azure SQL Databases. Enable the security contact with the operations email (from Key Vault)."
- Use `#mcp_azure_query_logs` to query the Defender for Cloud recommendations for the TravelReporting resource group and list any High or Critical severity items.
- Ask Copilot: "I have the following Defender for Cloud recommendations for TravelReporting: [paste findings]. Prioritise them by risk and suggest a remediation order."

- Review the Defender for Cloud Secure Score for the resource group and identify the top 3 controls that would most improve the score.
- Enable **just-in-time VM access** if any VMs are in scope (e.g. Bastion host from Phase 5).

GitHub Advanced Security:

- Enable Code Scanning (CodeQL) on the repository — ask Copilot: *"Generate a GitHub Actions / Azure DevOps CodeQL scanning workflow for a .NET 8 repository."*
 - If using Azure DevOps: use the Defender for DevOps integration or the CodeQL task from the Marketplace.
- Enable **Secret Scanning** and **Push Protection** on the repository — verify it would have caught the hardcoded email address from `TravelReporting/application.cfm`.
- Enable **Dependabot** alerts and security updates — note that the ColdFusion source has no package manager, but the migrated .NET 8 code (`TravelReporting.Api.csproj`) does.
- Triage any CodeQL findings on the migrated .NET code. Ask Copilot: *"CodeQL reported [finding] in TravelReporting.Api. Explain the risk and generate the fix."*

Well-Architected Framework review:

- Use Ask mode to conduct a WAF Security pillar self-assessment against the TravelReporting platform. Ask Copilot: *"Walk me through the Microsoft Azure Well-Architected Framework Security pillar design principles as they apply to a .NET 8 App Service application with Azure SQL, Key Vault, Azure Firewall, and Defender for Cloud. For each principle, tell me whether our current design addresses it, partially addresses it, or leaves a gap."*
- Document the gaps in `docs/waf-security-review.md`.

What Done Looks Like

- ☐ Defender for App Service and Defender for Azure SQL are enabled
- ☐ Defender for Cloud Secure Score is visible and at least 3 High findings are remediated
- ☐ CodeQL scanning is running on the repository (in Azure DevOps or GitHub Actions)
- ☐ Secret scanning and push protection are enabled — zero secrets in current HEAD
- ☐ Dependabot is enabled and initial alert triage is complete
- ☐ WAF Security pillar review is documented with identified gaps

☀ **Stretch Goal** — Export the Defender for Cloud regulatory compliance dashboard (MCSB) as a PDF and attach it to the architecture review minutes from the App Dev track. Identify the

top 5 MCSB controls that are currently failing.

Phase 8 — Operational & Security Dashboards + Handover Documentation

Agent mode

The final phase makes the platform visible. Engineers who inherit this system need to know at a glance: is it healthy, is it secure, and if something is wrong, what do they do? This phase produces Azure Monitor Workbooks for operational visibility, a Defender for Cloud security posture summary, and the handover documentation that ties it all together.

Your Challenge

Produce an operational Azure Monitor Workbook, a security posture summary, a trust-boundary architecture diagram, and complete handover documentation for the TravelReporting platform.

Things to Explore/Do

Operational dashboard:

- Use Agent mode: *"Generate an Azure Monitor Workbook JSON template for TravelReporting that shows: App Service request rate (200 vs 4xx vs 5xx), average response time per endpoint, App Insights availability test results, Azure SQL DTU over time, and active alert count."*
- Deploy the Workbook via Bicep (`infra/modules/monitoring.bicep`) — not manually through the portal.
- Use `@azure-resource-visualizer` to generate a visual map of the deployed resources and their connections.

Security posture summary:

- Query Defender for Cloud via the Azure MCP Server: `#mcp_azure_query_logs` — list all open security recommendations grouped by severity for the TravelReporting resource group.
- Ask Copilot: *"Generate a Markdown security posture summary for TravelReporting that lists: enabled Defender plans, Secure Score, open High/Critical findings, security controls in place (Key Vault, private endpoints, HTTPS-only, RBAC), and known residual risks."*

Architecture diagram:

- Use Agent mode: *"Generate a Mermaid diagram showing the TravelReporting target architecture: Azure DevOps pipeline → App Service (spoke VNet) → Azure SQL (private endpoint) and Key Vault (private endpoint), with Azure Firewall in the hub VNet controlling outbound traffic, Azure Monitor / App Insights receiving logs, and Defender for Cloud monitoring all resources. Mark trust boundaries with a dashed line."*

Handover documentation:

- Use Agent mode to generate `docs/handover.md` covering:
 - Resource inventory (all Azure resources, their names, SKUs, and environment)
 - Secrets inventory (what is in Key Vault, not the values — just the names and rotation schedule)
 - RBAC assignments (who/what has access to what)
 - Runbook index (linking to `runbooks/ops/` and `runbooks/secops/`)
 - Known gaps and outstanding risks (from the WAF review in Phase 7)
 - Contacts: App Dev owner, Platform owner, Security owner
- Include a RACI table for ongoing operations: Incident Response / Secret Rotation / Certificate Renewal / Dependency Updates / Security Alert Triage.

Final pipeline gate:

- Ask Copilot to add a `waf-check` stage to `azure-pipelines.yml` that runs the security smoke script from Phase 4 as a final gate before the production deployment is approved.
- Confirm the manual approval gate on the Azure DevOps `production` environment is configured.

What Done Looks Like

- ☐ Azure Monitor Workbook is deployed via Bicep and accessible in the portal
- ☐ Security posture summary document exists and is accurate
- ☐ Mermaid architecture diagram with trust boundaries is in the repository
- ☐ `docs/handover.md` exists with resource inventory, secrets inventory, RBAC list, and RACI
- ☐ Runbook index links all ops and SecOps runbooks
- ☐ Manual approval gate is configured on the `production` environment in Azure DevOps
- ☐ Security smoke script runs as a final pipeline gate

☀ **Stretch Goal** — Use the Azure MCP Server to run a cost analysis (`@azure-cost`) across all provisioned resources and produce a monthly cost estimate. Add a cost alert (Azure Budget) that emails the operations contact if the monthly spend exceeds the estimate by 20%.

▣ GIT CHECKPOINT

```
git add infra/ docs/ runbooks/ && git commit -m "phase-8: dashboards, handov
```

GHCP Prompting Cheat Sheet

💬 Ask Mode — Understanding & Reviewing

```
Review my infra/modules/app-service.bicep module. List every security harden
setting that is missing or not enforced, referencing the Azure Security Benc
for App Service (ASB-DP-1 through ASB-NS-4). For each gap, show the Bicep pr
that would fix it.
```

```
The TravelReporting application uses Windows NT integrated auth (CGI.AUTH_US
in TravelReporting/application.cfm. What are the security risks of this appr
in a cloud-hosted environment? What does the equivalent Microsoft Entra ID E
configuration look like for Azure App Service, and what claims would replace
CGI.AUTH_USER?
```

```
@azure-diagnostics Check the diagnostic settings for all resources in the
TravelReporting resource group and identify any resource that is NOT sending
logs to the Log Analytics workspace.
```

📋 Plan Mode — Designing

Design the private endpoint and DNS resolution architecture for an Azure App that needs to connect to Azure SQL and Azure Key Vault via private endpoints hub-spoke VNet topology. The hub VNet contains Azure Firewall. Show the DNS flow for a SQL connection from the App Service, the required private DNS zone and the Bicep resources needed for each step.

Design an alert strategy for TravelReporting that covers both operational and security events. The application is an internal government travel approval with ~200 users. Define: what to alert on, what NOT to alert on (noise reduction), severity levels, action group routing, and the on-call escalation path. Format as a decision table.

#mcp_azure_list_resources List all resources in the TravelReporting resource group. For each resource, tell me whether it has a public network access enabled, whether it has diagnostic settings configured, and whether it has a system-assigned Managed Identity. Highlight any that fail all three checks.

Agent Mode — Generating & Implementing

Generate a complete Bicep module (infra/modules/network.bicep) for a hub-spoke VNet topology for TravelReporting. Include: hub VNet (10.0.0.0/16) with AzureFirewallSubnet and AzureBastionSubnet, spoke VNet (10.1.0.0/16) with app-service integration subnet and private endpoints subnet, VNet peering both directions, private DNS zones for Azure SQL (privatelink.database.windows.net) and Key Vault (privatelink.vaultcore.azure.net), and private endpoints for both services in the spoke. Output valid Bicep with parameters for environment (dev, test, prod).

Generate a PowerShell smoke-test and security-check script for the TravelReporting App Service post-deployment. The script must: (1) verify GET /health returns 200, (2) verify HTTPS enforcement (HTTP redirects to HTTPS), (3) verify Strict-Transport-Security header is present, (4) verify the Azure SQL public network access is Disabled using the Azure CLI, (5) verify the Key Vault has

purge protection enabled. Exit with code 1 if any check fails.
Output suitable for Azure DevOps pipeline logging.

Generate an Azure DevOps pipeline YAML stage (smoke-test) that runs scripts/smoke/security-check.ps1 after the Deploy stage. The stage must: run on the Azure DevOps agent, use the service connection \$(AZURE_SERVICE_CONNECTION), fail the pipeline if the script exits non-zero, and publish the output as a pipeline artefact named security-smoke-results.

Security & SecOps Prompts

Perform a threat model for the TravelReporting application using the STRIDE methodology. The application is: a .NET 8 minimal API on Azure App Service, backed by Azure SQL (private endpoint), secrets in Key Vault, authenticated via Microsoft Entra ID, deployed via Azure DevOps, monitored by App Insights and Defender for Cloud. For each STRIDE category, list the top threat, the current control, and any

Review infra/main.bicep and all modules under infra/modules/ for security misconfigurations. Check for: public network access on PaaS resources, missing diagnostic settings, TLS versions below 1.2, Key Vault without soft-delete or purge protection, storage accounts with public blob access, RBAC assignments broader than necessary. Report each finding as:
[CRITICAL/HIGH/MEDIUM] | Resource | Issue | Bicep fix.

Write a KQL query for Azure Monitor / Log Analytics that detects the following suspicious behaviour on the TravelReporting App Service: (1) more than 10 requests to /travels returning HTTP 403 from the same IP in 5 minutes (possible authorisation bypass probe), (2) requests to /travels/{id} where the id parameter contains SQL metacharacters (' ; -- /*), (3) any request to /admin/* from an IP not in the DWER corporate IP range.

I have the following Defender for Cloud recommendations for TravelReporting (copy/paste the recommendation list from the portal or MCP output). Prioritise them by exploitability and business impact for a WA Government internal application. For each High or Critical finding, generate the Bicep or Azure CLI command that would remediate it.

Final Deliverables Checklist

1. ☐ **Secure IaC templates** — `infra/main.bicep` and all modules (`app-service.bicep` , `sql.bicep` , `keyvault.bicep` , `monitoring.bicep` , `network.bicep`) validated by Bicep MCP, deployed to at least one environment
2. ☐ **Identity + Key Vault + MI wiring** — all secrets in Key Vault, App Service reading via Key Vault references or Managed Identity, RBAC assignments documented, zero hardcoded credentials in repository
3. ☐ **Ops + security alert rules (KQL pack)** — at least 4 operational KQL alert rules and 4 security KQL alert rules deployed via Bicep, with action groups routing to correct contacts
4. ☐ **Health check + security smoke scripts** — `scripts/smoke/` contains operational and security smoke scripts, integrated as a post-deploy gate in `azure-pipelines.yml`
5. ☐ **Hub-spoke network with Azure Firewall** — Bicep for hub VNet, spoke VNet, Azure Firewall, private endpoints (Azure SQL + Key Vault), App Service VNet Integration, private DNS zones
6. ☐ **Incident runbooks + SecOps playbooks + auto-remediation** — 5 ops runbooks and 3 SecOps playbooks in `runbooks/` , at least 1 auto-remediation script (PowerShell or Logic App)
7. ☐ **Defender for Cloud + GitHub Advanced Security** — Defender plans enabled for App Service and Azure SQL, CodeQL scanning active, secret scanning and push protection enabled, Dependabot enabled, initial findings triaged
8. ☐ **WAF Security pillar self-assessment** — `docs/waf-security-review.md` with gap analysis across all Security pillar principles, Defender for Cloud Secure Score recorded
9. ☐ **Ops + security dashboards** — Azure Monitor Workbook deployed via Bicep, Defender for Cloud security posture summary documented
10. ☐ **Handover documentation + trust-boundary architecture diagram** — `docs/handover.md` with resource inventory, secrets inventory, RBAC table, RACI, and

Mermaid architecture diagram with trust boundaries marked

Tips for Success

1. **Prefer MCP over portal clicks.** Every action you take in the Azure portal needs to be in Bicep — if it is not in code, it did not happen. Use the Azure MCP Server and Azure Skills to query and inspect resources without leaving VS Code.
2. **Validate Bicep before deploying.** Use `mcp_bicep_get_best_practices` and `az deployment what-if` before every `az deployment group create`. A failed what-if is far cheaper than a failed deployment.
3. **Treat alerts AND detections as code.** KQL alert rules in Bicep are version-controlled, reviewable, and deployable. KQL rules created manually in the portal will be lost when the resource group is recreated.
4. **Runbooks and playbooks live next to the code.** A runbook in Confluence that nobody can find at 3am is not a runbook. Keep them in `runbooks/` in the repository, referenced from the Azure Monitor Workbook.
5. **Shift-left on security.** CodeQL, secret scanning, and push protection in the PR pipeline catch problems before they reach Azure. A secret caught by push protection never needs a rotation playbook.
6. **Least-privilege by default.** Start with no access and add what is needed — not the reverse. Every `Owner` role assignment is a future incident waiting to happen. Use `@azure-rbac` to audit and tighten assignments.
7. **Co-ordinate with the App Dev track.** The App Dev team owns `azure-pipelines.yml` and the application code. Your smoke scripts and security gates run in their pipeline — agree on the interface early (script exit codes, artefact names, environment names).
8. **Partial completion is fine.** A deployed, monitored, Key-Vault-wired App Service with private endpoints and one runbook is a better outcome than an undeployed perfect architecture diagram. Ship the infrastructure, then harden it incrementally.